Kontrol Motor DC via Komunikasi Serial (Python GUI ke ESP32)

Dosen Pengampu: Prof. Dr. Basuki Rahmat, S.Si., M.T.

Disusun Oleh:

Nama : Sanafa Mawaddah R. Z

NPM : 24081010153

PROGRAM STUDI INFORMATIKA FAKULTAS ILMU KOMPUTER

UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN" JAWA TIMUR

2026

A. Tujuan Modul

1. Mampu merancang sistem komunikasi data serial dua arah antara komputer (aplikasi antarmuka Python GUI) dan mikrokontroler ESP32 melalui kabel USB.
2. Memahami teknik manipulasi dan *parsing* data tipe String pada bahasa C++ (Arduino Framework) untuk mengekstrak perintah kontrol.
3. Mampu mengendalikan status aktif (ON/OFF) dan mengatur kecepatan (PWM) Motor DC berdasarkan perintah teks spesifik yang dikirim dari sistem eksternal.

B. Skema Rangkaian

Sistem ini membutuhkan integrasi antara ESP32 dan modul Motor Driver (misalnya L298N atau L293D) dengan konfigurasi jalur kabel sebagai berikut:

- Pin 27 (ESP32) dihubungkan ke IN1 Motor Driver sebagai pengontrol arah putaran maju.
- Pin 26 (ESP32) dihubungkan ke IN2 Motor Driver sebagai pengontrol arah putaran mundur.
- Pin 12 (ESP32) dihubungkan ke ENA Motor Driver sebagai penerima sinyal PWM untuk kontrol kecepatan.
- Output Motor Driver (OUT1 & OUT2) dihubungkan ke kutub positif dan negatif Motor DC.
- Pin GND ESP32 wajib digabung (di-common) dengan pin GND pada Motor Driver.

C. Konsep & Logika Sistem

Sistem ini menggunakan arsitektur komunikasi Serial *Master-Slave*. Aplikasi Python di komputer bertindak sebagai *Master* yang mengirimkan instruksi dalam bentuk teks (String) melalui port USB (COM Port). ESP32 bertindak sebagai *Slave* yang terus-menerus memantau antarmuka Serial.

Ketika ada aliran data masuk, ESP32 membaca seluruh karakter hingga mendeteksi karakter *Enter* (\n). Teks yang ditangkap kemudian dibersihkan dari spasi atau karakter tersembunyi menggunakan fungsi trim(). Setelah itu, program menggunakan struktur logika kondisional (if-else) untuk mencocokkan teks tersebut dengan daftar perintah yang dikenali. Jika cocok, ESP32 langsung mengeksekusi perubahan tegangan pada pin GPIO dan membalas dengan pesan konfirmasi ke Python.

D. Daftar Perintah

Format instruksi string yang dikenali oleh sistem ESP32:

- 1 : Mengaktifkan motor secara otomatis ke kecepatan default (Duty Cycle PWM: 200).
- 0 : Mematikan motor secara total (mengubah semua pin ke logika LOW dan PWM ke 0).
- PWM:xxx : Mengubah nilai kecepatan motor secara spesifik. Nilai xxx adalah angka antara 0 hingga 255 (contoh: PWM:125).

E. Source Code dan Penjelasan Logika Program

1. Source code phyton (Thinker GUI)

```
import tkinter as tk
from tkinter import ttk, messagebox
import serial
import serial.tools.list_ports

class MotorControllerApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Python ESP32 Motor Controller")
        self.root.geometry("400x380")
        self.root.resizable(False, False)

        self.serial_port = None

        self.create_widgets()
        self.refresh_ports()

    def create_widgets(self):
        # Frame Koneksi Serial
        conn_frame = ttk.LabelFrame(self.root,
text="Pengaturan Koneksi Serial")
        conn_frame.pack(pady=10, padx=10, fill="x")

        self.port_var = tk.StringVar()
        self.port_cb = ttk.Combobox(conn_frame,
textvariable=self.port_var, state="readonly", width=15)
        self.port_cb.grid(row=0, column=0, padx=10,
pady=10)

        btn_refresh = ttk.Button(conn_frame, text="Refresh
Port", command=self.refresh_ports)
        btn_refresh.grid(row=0, column=1, padx=5, pady=10)

        self.btn_connect = ttk.Button(conn_frame,
text="Connect", command=self.toggle_connection)
        self.btn_connect.grid(row=0, column=2, padx=5,
pady=10)

        # Frame Kontrol Motor Dasar (ON/OFF)
        ctrl_frame = ttk.LabelFrame(self.root,
```

```
text="Kontrol Motor Dasar")
        ctrl_frame.pack(pady=10, padx=10, fill="x")

        btn_on = tk.Button(ctrl_frame, text="MOTOR ON",
bg="#4CAF50", fg="white", font=("Arial", 10, "bold"),
command=lambda: self.send_command("1"))
        btn_on.pack(side="left", expand=True, fill="x",
padx=10, pady=15)

        btn_off = tk.Button(ctrl_frame, text="MOTOR OFF",
bg="#F44336", fg="white", font=("Arial", 10, "bold"),
command=lambda: self.send_command("0"))
        btn_off.pack(side="right", expand=True, fill="x",
padx=10, pady=15)

        # Frame Kecepatan (PWM)
        pwm_frame = ttk.LabelFrame(self.root,
text="Kontrol Kecepatan (PWM)")
        pwm_frame.pack(pady=10, padx=10, fill="x")

        self.pwm_slider = tk.Scale(pwm_frame, from_=0,
to=255, orient="horizontal", tickinterval=51)
        self.pwm_slider.set(150) # Set nilai awal ke 150
        self.pwm_slider.pack(fill="x", padx=10, pady=5)

        btn_set_pwm = ttk.Button(pwm_frame, text="Set
Kecepatan", command=self.set_pwm)
        btn_set_pwm.pack(pady=10)

        # Status Label di bagian bawah
        self.status_label = ttk.Label(self.root,
text="Status: Disconnected", foreground="red",
font=("Arial", 10, "bold"))
        self.status_label.pack(side="bottom", pady=10)

    def refresh_ports(self):
        """Mencari semua Port COM (Kabel USB) yang
tersedia di laptop"""
        ports = serial.tools.list_ports.comports()
        self.port_cb['values'] = [port.device for port in
ports]
        if self.port_cb['values']:
            self.port_cb.current(0)

    def toggle_connection(self):
        """Menghubungkan atau memutus koneksi Serial
dengan ESP32"""
        if self.serial_port and self.serial_port.is_open:
            self.serial_port.close()
            self.btn_connect.config(text="Connect")
            self.status_label.config(text="Status:
Disconnected", foreground="red")
            self.port_cb.config(state="readonly")
        else:
            port = self.port_var.get()
            if port:
                try:
```

```
                    # Baudrate 115200 harus sama dengan
Serial.begin di ESP32
                    self.serial_port = serial.Serial(port,
115200, timeout=1)

self.btn_connect.config(text="Disconnect")

self.status_label.config(text=f"Status: Connected to
{port}", foreground="green")
                    self.port_cb.config(state="disabled")
                except Exception as e:
                    messagebox.showerror("Error", f"Gagal
terhubung ke {port}: {str(e)}\n\nPastikan Serial Monitor
di Arduino IDE sudah ditutup!")
            else:
                messagebox.showwarning("Peringatan",
"Pilih port COM terlebih dahulu!")

    def send_command(self, cmd):
        """Mengirimkan teks perintah ke ESP32"""
        if self.serial_port and self.serial_port.is_open:
            try:
                # Mengirim data dengan tambahan newline
(\n) agar bisa dibaca oleh Serial.readStringUntil
                self.serial_port.write((cmd +
'\n').encode('utf-8'))
            except Exception as e:
                messagebox.showerror("Error", f"Gagal
mengirim data: {str(e)}")
        else:
            messagebox.showwarning("Peringatan", "Silakan
klik tombol 'Connect' terlebih dahulu!")

    def set_pwm(self):
        """Mengambil nilai slider dan mengirimkannya ke
ESP32 dengan format PWM:xxx"""
        val = self.pwm_slider.get()
        self.send_command(f"PWM:{val}")

if __name__ == "__main__":
    root = tk.Tk()
    app = MotorControllerApp(root)
    root.mainloop()
```

2. Source code C++ (ESP32 Receiver)

```
// KODE ESP32 UNTUK MENERIMA PERINTAH DARI PYTHON GUI
int motor1Pin1 = 27;
int motor1Pin2 = 26;
int enable1Pin = 12;

const int freq = 30000;
const int resolution = 8;
```

```
void setup() {
  Serial.begin(115200);
  pinMode(motor1Pin1, OUTPUT);
  pinMode(motor1Pin2, OUTPUT);
  ledcAttach(enable1Pin, freq, resolution);

  // Pastikan motor mati saat awal menyala
  digitalWrite(motor1Pin1, LOW);
  digitalWrite(motor1Pin2, LOW);
  ledcWrite(enable1Pin, 0);

  Serial.println("Sistem Siap! Menunggu perintah dari
Python GUI...");
}

void loop() {
  // Mengecek apakah ada data yang masuk dari kabel USB
(Python)
  if (Serial.available() > 0) {
    String command = Serial.readStringUntil('\n');
    command.trim(); // Menghapus spasi atau karakter enter
berlebih

    if (command == "1") {
      digitalWrite(motor1Pin1, HIGH);
      digitalWrite(motor1Pin2, LOW);
      ledcWrite(enable1Pin, 200); // Kecepatan default
saat ON
      Serial.println(">>> Motor Dinyalakan");

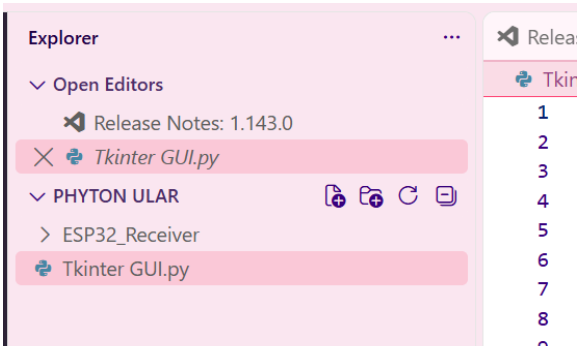
    } else if (command == "0") {
      digitalWrite(motor1Pin1, LOW);
      digitalWrite(motor1Pin2, LOW);
      ledcWrite(enable1Pin, 0);
      Serial.println(">>> Motor Dimatikan");

    } else if (command.startsWith("PWM:")) {
      String pwmValueStr = command.substring(4); //
Mengambil angka setelah teks "PWM:"
      int pwmValue = pwmValueStr.toInt();
      ledcWrite(enable1Pin, pwmValue);
```

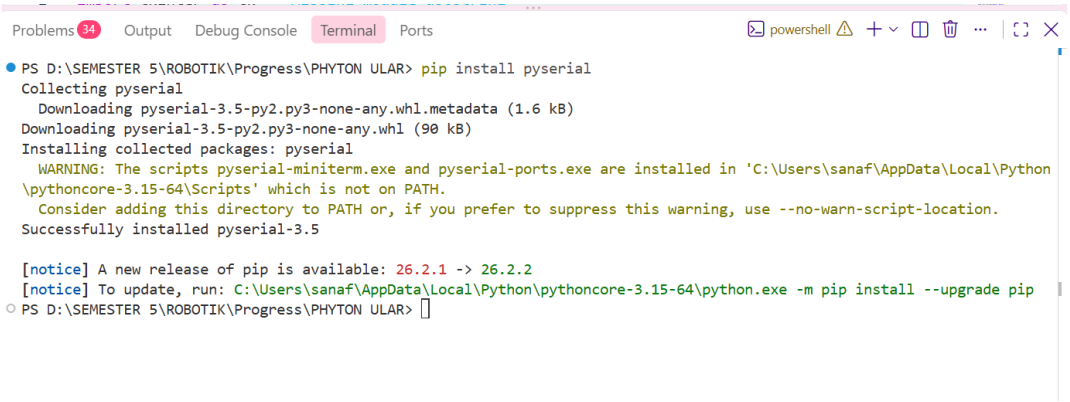
```
        Serial.print(">>> Kecepatan (PWM) diubah menjadi:
");
        Serial.println(pwmValue);
    }
  }
}
```

F. Hasil Pengujian & Analisis

1. Proses Kompilasi dan Koneksi Serial

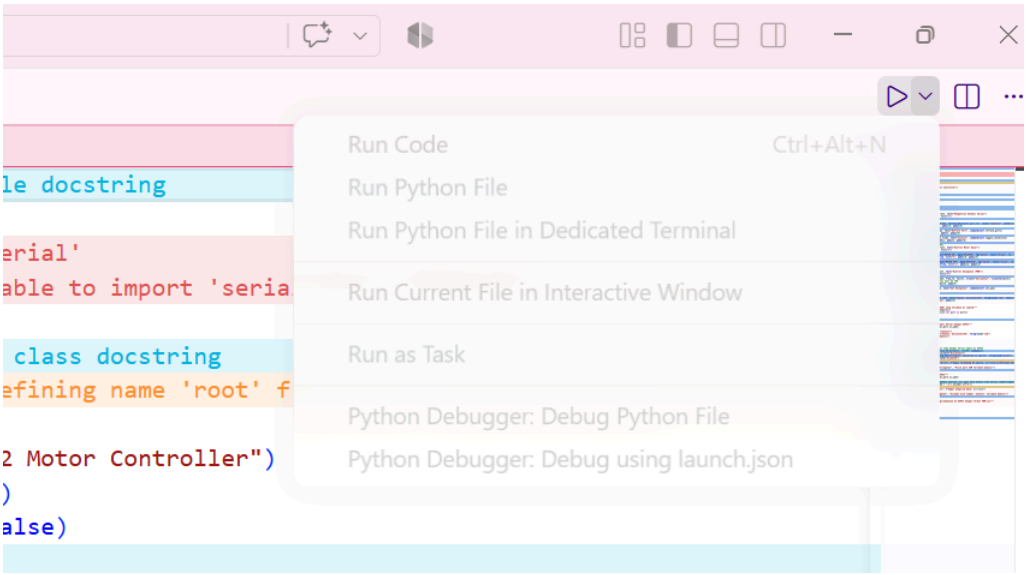


Langkah pertama adalah menyiapkan ruang kerja (*workspace*) di Visual Studio Code. Pastikan file skrip Python untuk antarmuka (Tkinter GUI.py) dan folder proyek C++ untuk mikrokontroler (ESP32_Receiver) berada dalam satu direktori kerja yang sama agar mudah dikelola.

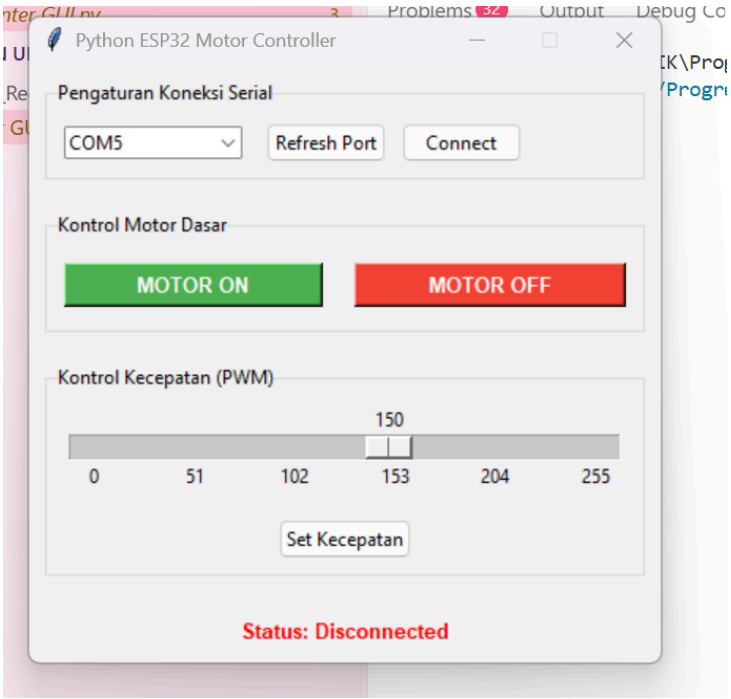


Agar Python dapat melakukan komunikasi data dengan port USB (Serial), kita wajib menginstal pustaka eksternal. Melalui terminal VS Code, jalankan perintah pip install pyserial. Gambar menunjukkan bahwa pustaka *pyserial* telah sukses diunduh dan dipasang pada *environment* Python lokal.

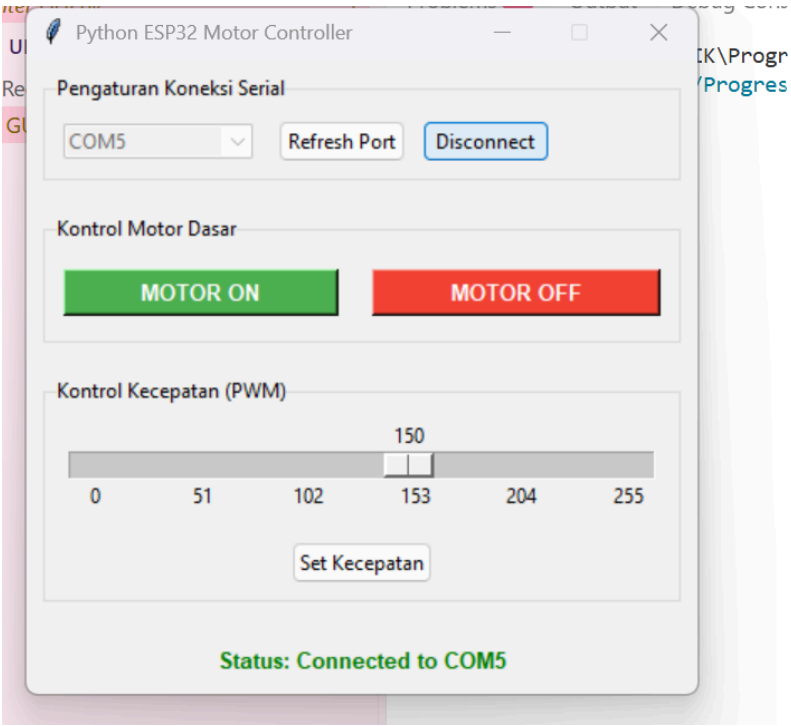
2. Pengujian Respon Aktuator Motor



Skrip Python dieksekusi dengan mengklik kanan editor dan memilih opsi "Run Python File in Dedicated Terminal". Jendela aplikasi "Python ESP32 Motor Controller" akan muncul dengan status awal berwarna merah "Status: Disconnected", menandakan GUI belum terhubung ke port mikrokontroler.



Pengguna memilih port yang sesuai (COM5) dari *dropdown menu* lalu menekan tombol "Connect". Tulisan di bagian bawah berubah menjadi hijau "Status: Connected to COM5", yang membuktikan bahwa jembatan komunikasi serial antara komputer dan ESP32 telah berhasil dibuka.



Antarmuka GUI memungkinkan pengguna memberikan perintah tanpa mengetik teks. Saat *slider* "Kontrol Kecepatan (PWM)" digeser ke kanan (misalnya ke angka 255) lalu tombol "Set Kecepatan" ditekan, program Python di belakang layar akan merangkai string perintah "PWM:255" dan menembakkannya ke ESP32 melalui komunikasi Serial. *(Catatan untukmu: Di gambar ini statusnya tercatat "Disconnected" mungkin karena kamu tidak sengaja menekan tombol putus koneksi saat mengambil screenshot, tapi penjelasannya bisa tetap difokuskan pada fungsi slider-nya).*

3. Hasil Analisis

Berdasarkan pengujian yang dilakukan, integrasi antara antarmuka GUI Python (Tkinter) dan ESP32 melalui komunikasi Serial berjalan dengan sangat baik dan stabil. Penggunaan GUI secara signifikan mempermudah interaksi pengguna karena instruksi *hardware* tidak perlu lagi diketik manual melalui Serial Monitor Arduino IDE.

Pustaka pyserial terbukti andal dalam mengirimkan paket data *string* secara instan (*real-time*). Ketika pengguna menekan tombol "MOTOR ON/OFF" atau menggeser nilai *slider* PWM di layar komputer, skrip Python langsung menerjemahkannya menjadi format teks yang disepakati (seperti "1", "0", atau "PWM:xxx"). Di sisi lain, mikrokontroler ESP32 juga berhasil menangkap teks tersebut, membuang spasi kosong (*trimming*), melakukan

ekstraksi angka (*parsing*), dan mengeksekusinya menjadi perubahan sinyal listrik (PWM) pada pin ENA Motor Driver. Kunci keberhasilan sistem ini terletak pada kesamaan pengaturan *baud rate* serta penentuan format *newline* (\n) sebagai penanda akhir instruksi.